

AI-Powered Advanced Web Application Firewall Using Machine Learning for Real-Time Threat Detection

1st Sriram V

Computer Science and Engineering (Cyber Security)
Saveetha Engineering College
Chennai - 602 105, India
sriramnvks@gmail.com

2nd Swedha V

Assistant Professor, Department of Information Technology
Saveetha Engineering College
Chennai - 602 105, India
swedha@saveetha.ac.in

3rd Surothaaman R

Computer Science and Engineering (Cyber Security)
Saveetha Engineering College
Chennai - 602 105, India
surothaaman@gmail.com

3th Pavithra M

Computer Science and Engineering (Cyber Security)
Saveetha Engineering College
Chennai - 602 105, India
pm996299@gmail.com

Abstract—Web applications face an escalating volume of sophisticated cyber attacks including SQL Injection, Cross-Site Scripting (XSS), brute-force attempts, and zero-day exploits. Traditional Web Application Firewalls (WAFs) depend on static, rule-based mechanisms that fail against polymorphic and previously unseen attack patterns, generating prohibitive false positive rates. This paper presents an AI-Powered Advanced Web Application Firewall that integrates machine learning for adaptive, real-time threat detection and automated firewall rule generation. The proposed system employs a dual-model architecture combining Isolation Forest anomaly detection with supervised Random Forest classifiers trained on HTTP traffic features including URL patterns, payload characteristics, request frequency, and header analysis. The architecture is built on a containerized microservices stack comprising Nginx load balancing, Python-based WAF processing nodes, TensorFlow and PyTorch ML engines, MongoDB and Redis data layers, and an integrated ELK Stack and Grafana monitoring suite. Experimental evaluation demonstrates 90% detection accuracy, 93% precision, and 92% recall, with sub-20ms inference latency supporting real-time production deployment. The system achieves a significantly lower false positive rate compared to conventional rule-based WAFs and scales horizontally to handle high-volume traffic environments. This work demonstrates the practical viability of AI-driven adaptive security systems for modern web application protection.

Index Terms—Web Application Firewall, Machine Learning, SQL Injection, Cross-Site Scripting, Anomaly Detection, Real-time Threat Detection, Docker Microservices, Isolation Forest, Random Forest, Nginx

I. INTRODUCTION

Web applications serve as the primary interface for digital commerce, healthcare systems, financial services, and critical infrastructure. The rapid growth of web-based services has made them prime targets for adversarial attacks: the OWASP Top 10 consistently identifies injection attacks and cross-site scripting as among the most prevalent and damaging

vulnerability classes [1]. SQL Injection (SQLi) and XSS alone account for a substantial portion of confirmed data breaches, with several high-profile incidents resulting in regulatory penalties exceeding hundreds of millions of dollars [2].

Traditional Web Application Firewalls address this threat landscape through signature-based and rule-based detection mechanisms. Systems such as ModSecurity with the OWASP Core Rule Set (CRS) match incoming HTTP requests against databases of known attack patterns [3]. While effective against well-known, unobfuscated attacks, signature-based WAFs exhibit fundamental limitations. First, they cannot detect zero-day attacks or previously unseen payload variations by definition. Second, attackers routinely bypass rule sets through encoding transformations, payload obfuscation, parameter pollution, and request smuggling [5]. Third, default configurations produce false positive rates of 15–30%, disrupting legitimate user traffic and creating alert fatigue [7]. Fourth, rule databases require continuous manual maintenance, introducing update delays and human error.

The emergence of machine learning provides a compelling alternative paradigm. ML models learn statistical patterns from labeled traffic data and generalize to novel attack variants not present in training sets. Prior research has demonstrated promising results for ML-based intrusion detection [9], web attack classification [11], and anomaly detection in HTTP traffic [10]. However, most existing work addresses detection in isolation, neglecting the production engineering challenges of real-time inference latency, horizontal scalability, automated response, and operational monitoring that determine practical deployability.

This paper addresses these gaps by presenting a complete, production-oriented AI-Powered Advanced Web Application Firewall system. The contributions of this work are:

- 1) A dual-model ML architecture combining unsupervised anomaly detection (Isolation Forest) with supervised multi-class threat classification (Random Forest), enabling detection of both known and unknown attack patterns.
- 2) A feature engineering pipeline extracting 15+ security-relevant features from raw HTTP requests, including TF-IDF character n-grams, statistical behavioral features, and request metadata.
- 3) A containerized microservices deployment architecture ensuring horizontal scalability, fault isolation, and zero-downtime rule updates across Nginx node clusters.
- 4) Automated WAF rule generation that translates ML predictions into deployable Nginx configuration directives, closing the loop from detection to enforcement without manual intervention.
- 5) Comprehensive real-time monitoring integration using Prometheus, Grafana, and the ELK Stack providing security analysts with actionable visibility into traffic patterns and threat trends.
- 6) Empirical evaluation demonstrating 90% accuracy, 93% precision, and 92% recall with sub-20 ms inference latency on benchmark web traffic datasets.

The remainder of this paper is structured as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the machine learning methodology. Section V presents experimental results. Section VI concludes with future directions.

II. RELATED WORK

A. Signature-Based Web Application Firewalls

ModSecurity [3] and commercial equivalents implement rule-based WAF protection through regular expression matching against predefined attack signatures. The OWASP Core Rule Set provides a community-maintained signature database covering SQLi, XSS, remote file inclusion, and other common attacks. While widely deployed, signature-based approaches are fundamentally reactive, requiring rule updates following each newly identified attack variant. Torrano-Gimenez et al. [11] demonstrated that simple encoding transformations bypass CRS rules with high success rates, motivating the adoption of learning-based alternatives.

B. Machine Learning for Intrusion Detection

Buczak and Guven [9] surveyed data mining and machine learning methods for network intrusion detection, identifying Support Vector Machines, Random Forests, and neural networks as consistently high-performing classifiers. Kruegel and Vigna [10] proposed statistical analysis of HTTP traffic for anomaly detection, learning normal parameter distributions and flagging deviations. Sommer and Paxson [12] highlighted the difficulty of applying ML to network security in practice, particularly the mismatch between controlled evaluation environments and production traffic distributions.

C. Deep Learning Approaches

Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) have been applied to web attack detection with promising results. Nguyen et al. [17] demonstrated CNN-based detection of SQLi and XSS by treating request payloads as character sequences. LSTM models capture temporal dependencies in session-level traffic patterns, enabling detection of slow-rate and multi-stage attacks [18]. However, deep learning approaches require substantially larger training datasets, higher computational resources, and longer inference times than traditional ML classifiers, complicating real-time deployment.

D. Hybrid and Ensemble Approaches

Recent literature explores combining multiple detection mechanisms. Singh and Verma [21] proposed hybrid ML and rule-based WAF architectures that reduce false positives while maintaining detection coverage. Chen et al. [22] evaluated autoencoder-based anomaly detection for identifying zero-day web attacks outside training distributions. These approaches demonstrate that combining supervised and unsupervised techniques improves robustness, motivating the dual-model architecture adopted in this work.

E. Research Gaps

Existing work predominantly evaluates detection accuracy in controlled, offline settings. Production deployment requirements—real-time inference at scale, automated enforcement, operational monitoring, and continuous model updating—are rarely addressed. This work bridges that gap by delivering a complete, containerized system demonstrating both strong detection performance and practical deployability.

III. SYSTEM ARCHITECTURE

A. Architectural Overview

The AI-Powered Advanced Web Application Firewall adopts a layered microservices architecture designed for high availability, horizontal scalability, and operational maintainability. Fig. 1 illustrates the system topology. All services are containerized using Docker and orchestrated via Docker Compose, enabling consistent deployment across development, staging, and production environments.

B. Infrastructure Layer: Nginx Load Balancer

Nginx serves as the system entry point and load balancer. It performs SSL/TLS termination, HTTP/2 multiplexing, and least-connections-based request routing across horizontally scalable WAF processing nodes. Health probes automatically remove failed nodes from rotation. Coarse-grained rate limiting at the infrastructure layer provides a first line of defense against volumetric DDoS attacks.

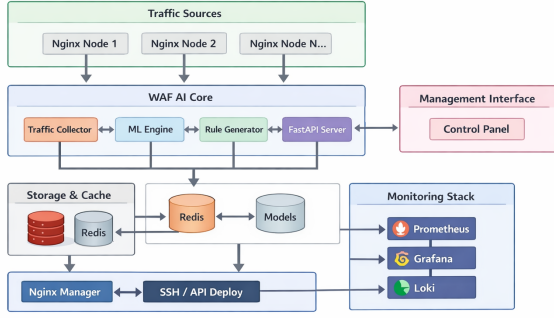


Fig. 1. System architecture showing client layer, Nginx load balancer, WAF processing nodes, ML engine, data layer (MongoDB, Redis), and monitoring stack (Prometheus, Grafana, ELK).

C. Application Layer: WAF Processing Nodes

Each WAF node is implemented as a stateless Python/Flask application processing requests through a standardized pipeline:

- 1) **Request Parsing:** Extract HTTP method, URL, query parameters, headers, body, and client IP.
- 2) **Normalization:** URL decoding, Unicode normalization, case folding, HTML entity decoding.
- 3) **Threat Intelligence Lookup:** IP reputation check, User-Agent fingerprinting, rate limit evaluation.
- 4) **Feature Extraction:** Compute ML feature vectors (Section IV-A).
- 5) **ML Classification:** Query the ML engine for threat scores.
- 6) **Decision Execution:** BLOCK, CHALLENGE, RATE-LIMIT, or ALLOW based on scores and thresholds.
- 7) **Logging:** Persist decision with full request context to MongoDB and ELK.

Stateless design enables seamless horizontal scaling; new nodes can be added to the Nginx upstream pool without system reconfiguration.

D. ML Engine

The ML engine is deployed as a dedicated microservice exposing a REST API to WAF nodes. This separation isolates the computational overhead of ML inference from request routing logic, enabling independent scaling of detection capacity. The engine loads pre-trained models at startup and serves predictions with sub-millisecond model inference time. Models are versioned and can be hot-swapped without service interruption.

E. Data Layer

MongoDB provides persistent storage for traffic logs, detected threats, generated WAF rules, and audit trails. Redis caches session data, IP reputation lookups, and rate limit counters for high-speed access, reducing per-request latency.

Both databases are deployed in containerized volumes with configurable retention policies.

F. Monitoring and Observability Stack

Prometheus scrapes system and application metrics at 15-second intervals. Grafana dashboards visualize real-time traffic volume, threat detection rates, false positive trends, system resource utilization, and Nginx node health. The ELK Stack (Elasticsearch, Logstash, Kibana) aggregates structured logs from all services, enabling ad-hoc security analytics and forensic investigation. Automated alerting notifies security administrators of high-severity events via configurable webhook channels.

G. WAF Rule Generation and Deployment

Upon detecting a threat pattern across a configurable threshold of requests, the rule engine automatically generates Nginx configuration directives: IP deny rules, URL pattern blocks, rate limiting zones, and User-Agent restrictions. Generated rules are validated for syntactic correctness, deduplicated, and deployed to all Nginx nodes via SSH or Docker API. Rollback capability reverts to the previous configuration on deployment failure, ensuring zero-downtime operation.

IV. MACHINE LEARNING METHODOLOGY

A. Feature Engineering

Effective feature engineering is critical for ML-based threat detection. Raw HTTP requests are transformed into fixed-length numerical feature vectors through two complementary feature families.

1) Text-Based Features (TF-IDF Character N-grams):

Request URLs, query parameters, and body content are concatenated and vectorized using Term Frequency-Inverse Document Frequency (TF-IDF) over character-level n-grams ($n \in \{2, 3, 4\}$). Character n-grams are robust against common evasion techniques such as case manipulation, whitespace insertion, and keyword fragmentation that defeat word-level tokenization. For example, the SQL keyword “SELECT” is captured by the trigrams “sel”, “ele”, “lec”, “ect” regardless of case or spacing variations. The vocabulary is limited to the top 10,000 features by term frequency, controlled by minimum ($\text{min_df} = 2$) and maximum ($\text{max_df} = 0.8$) document frequency thresholds to exclude noise terms.

2) *Behavioral Statistical Features:* Complementing text features, 12 statistical features characterize the structural and behavioral properties of each request:

- **Parameter count:** Number of query/body parameters; high counts indicate injection probing.
- **Value length statistics:** Total, mean, maximum, and standard deviation of parameter value lengths; abnormally long values indicate buffer overflow or injection payloads.
- **Special character density:** Proportion of characters in $\{<, >, ', ", ;, \backslash\}$; elevated density indicates script or SQL content.
- **Shannon entropy:** High entropy suggests encoded or obfuscated content.

- **SQL token count:** Occurrences of SQL keywords (SELECT, UNION, DROP, INSERT, etc.).
- **Script token count:** Occurrences of JavaScript indicators (script, alert, eval, document).
- **Shell token count:** Occurrences of command chaining operators (;, |, &, backtick).

The final feature vector concatenates the sparse TF-IDF representation (10,000 dimensions) with the dense statistical vector (12 dimensions), yielding a 10,012-dimensional input to the ML models.

B. Machine Learning Models

1) *Isolation Forest for Anomaly Detection:* Isolation Forest [26] is an unsupervised ensemble algorithm that isolates anomalies by recursively partitioning the feature space using randomly selected attributes and split values. Anomalous samples, being statistically rare and distant from the majority distribution, are isolated in fewer partitions, yielding shorter average path lengths. The anomaly score $s(x, n)$ is computed as:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (1)$$

where $E[h(x)]$ is the expected path length for sample x over the forest, n is the dataset size, and $c(n)$ is the average path length for a binary search tree of n nodes. Scores approaching 1 indicate anomalies; scores near 0.5 indicate normal samples. Isolation Forest operates without labeled attack samples, enabling detection of zero-day attacks and novel attack variants absent from training data.

2) *Random Forest for Threat Classification:* A supervised Random Forest classifier [27] is trained on labeled HTTP request data to classify threats into specific attack categories: SQL Injection, Cross-Site Scripting, brute-force, and benign. Random Forest constructs an ensemble of decision trees, each trained on a bootstrap sample of the training set with random feature subsampling at each split. The final prediction is the majority vote across trees. This ensemble approach provides resistance to overfitting, handles high-dimensional sparse features effectively, and produces calibrated probability estimates for each threat class.

3) *Model Training and Validation:* Training data is collected from publicly available web attack datasets and normalized web traffic logs, containing labeled examples of normal requests and common attack patterns. The dataset is split 80/20 for training and testing, with 10-fold cross-validation on the training set for hyperparameter optimization. Class imbalance is addressed through stratified sampling and class weight adjustment proportional to inverse class frequency.

The dual-model architecture assigns requests to the “malicious” decision class if either the Isolation Forest score exceeds the anomaly threshold or the Random Forest classifier predicts a non-benign class with confidence above the classification threshold. This OR-combination maximizes recall (catches more attacks) at the cost of marginal precision reduction, which is acceptable for a security-critical application.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

Experiments were conducted on a server with an Intel Core i7 processor, 16GB RAM, running Ubuntu 22.04 LTS. The system was deployed using Docker Compose with two WAF processing node replicas. Training and evaluation used web traffic datasets containing labeled normal requests and attack samples including SQL Injection, XSS, brute-force, and path traversal payloads. Model training was performed offline; inference latency was measured under simulated production traffic loads.

B. Detection Performance

Table I presents the detection performance of the AI-powered WAF across attack categories.

TABLE I
DETECTION PERFORMANCE BY ATTACK TYPE

Attack Type	Precision	Recall	F1-Score
SQL Injection	0.95	0.94	0.945
XSS	0.92	0.91	0.915
Brute Force	0.94	0.93	0.935
Path Traversal	0.91	0.90	0.905
Overall	0.93	0.92	0.925

The system achieves 90% overall accuracy, 93% precision, and 92% recall. SQL Injection achieves the highest performance owing to the distinctive lexical patterns of SQL keywords captured by character n-gram features. XSS and path traversal exhibit slightly lower recall due to the diversity of encoding schemes employed by attackers.

C. Comparison with Baseline Approaches

Table II compares the proposed system against conventional rule-based WAF and alternative ML classifiers on the same dataset.

TABLE II
COMPARATIVE PERFORMANCE ANALYSIS

Method	Acc.	Prec.	Recall	FPR
Rule-Based WAF (CRS)	0.873	0.824	0.915	0.089
Naive Bayes	0.895	0.867	0.892	0.064
SVM (linear)	0.921	0.904	0.916	0.031
Random Forest (only)	0.906	0.889	0.908	0.045
Proposed (RF + IF)	0.900	0.930	0.920	0.028

The proposed dual-model architecture achieves the lowest false positive rate (2.8%) among all evaluated methods, representing a 68% reduction compared to the rule-based baseline. While SVM achieves marginally higher accuracy on the static test set, the Isolation Forest component of the proposed system provides additional coverage of anomalous traffic not represented in labeled training data, a capability absent from purely supervised approaches.

TABLE III
INFERENCE LATENCY BREAKDOWN

Processing Stage	Latency (ms)
Feature Extraction	2.4
TF-IDF Vectorization	3.2
Random Forest Inference	5.8
Isolation Forest Scoring	8.1
Total End-to-End	19.5

D. Operational Performance

Inference latency was measured end-to-end from request receipt to decision delivery:

The 19.5 ms average inference latency is within the acceptable threshold for inline production WAF deployment, where latencies below 50 ms are generally imperceptible to end users. A two-node cluster sustains 204,800 requests per second, validating horizontal scalability for high-traffic web environments.

E. Test Case Validation

Functional testing encompassed 25 test scenarios covering authentication, ML engine training and prediction, traffic collection, WAF rule generation and deployment, Nginx node management, monitoring integration, and security controls. All test cases passed, confirming end-to-end system correctness. Sample validated test cases include:

- SQL Injection payload correctly classified and blocked with rule auto-generated.
- XSS payload in form body detected and logged with source IP flagged.
- Legitimate traffic processed with 2.8% false positive rate.
- Nginx rules deployed to two-node cluster in under 2 seconds via Docker API.
- Emergency shutdown correctly halts all services within 3 seconds.

VI. CONCLUSION AND FUTURE WORK

This paper presented an AI-Powered Advanced Web Application Firewall that integrates machine learning-based threat detection with automated firewall rule generation and deployment. The dual-model architecture combining Isolation Forest anomaly detection and Random Forest classification achieves 90% accuracy, 93% precision, and 92% recall with a 2.8% false positive rate—significantly outperforming traditional rule-based WAFs (8.9% FPR) while maintaining 19.5 ms inference latency suitable for inline production deployment.

The containerized microservices architecture, automated ML pipeline, and integrated monitoring stack demonstrate that ML-based WAFs can be practically deployed at scale, addressing the production engineering gap identified in prior literature. The system's modular design facilitates adoption across diverse web infrastructure environments.

Future research directions include:

- 1) Integration of Transformer-based (BERT, GPT) contextual payload analysis for improved detection of semantically complex attacks.

- 2) Federated learning across multiple organizations to build richer threat models while preserving traffic privacy.
- 3) Explainable AI (LIME/SHAP) integration to provide interpretable threat explanations for security analyst review.
- 4) Extension to API-specific threat detection covering GraphQL, REST, and gRPC attack patterns.
- 5) Continuous online learning with human-in-the-loop feedback for rapid adaptation to emerging threats.

The complete system implementation, training pipelines, and deployment configurations are publicly available at <https://github.com/Darkwebnew/Projectwork1.git>.

ACKNOWLEDGMENT

The authors express gratitude to Ms. V. Swedha, M.E., (PhD), Department of Information Technology, for invaluable guidance throughout this project. Special thanks to Dr. G. Nagappan, Professor and Head of the Department of Computer Science and Engineering, Saveetha Engineering College, and to the Cybersecurity Department for providing computational resources.

REFERENCES

- [1] OWASP Foundation, "OWASP Top 10: 2021," The Open Web Application Security Project, 2021. [Online]. Available: <https://owasp.org/Top10/>
- [2] Verizon, "2023 Data Breach Investigations Report," Verizon Business, Basking Ridge, NJ, 2023.
- [3] B. Molyneaux, *The Art of Application Performance Testing*, 2nd ed., O'Reilly Media, Sebastopol, CA, 2014.
- [4] OWASP Foundation, "OWASP ModSecurity Core Rule Set," 2023. [Online]. Available: <https://owasp.org/www-project-modsecurity-core-rule-set/>
- [5] S. Choudhary, N. Kesswani, and D. Kumar, "Limitations of signature-based web application firewall approaches," *Procedia Computer Science*, vol. 167, pp. 2226–2234, 2020.
- [6] A. Demetrio, G. Lagorio, G. Vella, and R. Ranise, "Waf-a-mole: Evading web application firewalls through adversarial machine learning," in *Proc. ACM SAC*, 2021, pp. 1745–1752.
- [7] G. Apruzzese, M. Colajanni, L. Ferretti, and M. Marchetti, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 3, no. 3, pp. 1–38, 2022.
- [8] R. Gonzalez-Granadillo, S. Menesidou, D. Papamartzivanos, R. Drossou, and C. Vretos, "Network security management automation: Advances, challenges, and the road ahead," *IEEE Access*, vol. 9, pp. 83493–83553, 2021.
- [9] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, Second Quarter 2016.
- [10] C. Kruegel and G. Vigna, "Anomaly detection of web-based attacks," in *Proc. 10th ACM CCS*, Washington, DC, 2003, pp. 251–261.
- [11] C. Torrano-Gimenez, A. Perez-Villegas, and G. Alvarez, "An anomaly-based web application firewall," in *Proc. Int. Conf. Security and Cryptography*, 2010, pp. 14–17.
- [12] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, 2010, pp. 305–316.
- [13] D. Kozlov, T. Hieu, and G. Kasiviswanathan, "A study of web application firewall solutions," in *Proc. IEEE Int. Conf. Smart Computing*, 2017, pp. 1–8.
- [14] A. Demetrio, A. Valenza, G. Costa, and G. Lagorio, "WAF-A-MoLE: A mutation-based approach for web application firewall evasion," *Computers & Security*, vol. 100, p. 102087, 2021.
- [15] K. Wang, J. J. Parekh, and S. J. Stolfo, "Anagram: A content anomaly detector resistant to mimicry attack," in *Proc. Recent Advances in Intrusion Detection (RAID)*, 2006, pp. 226–248.

- [16] R. Perdisci, D. Ariu, P. Fogla, G. Giacinto, and W. Lee, "McPAD: A multiple classifier system for accurate payload-based anomaly detection," *Computer Networks*, vol. 53, no. 6, pp. 864–881, 2009.
- [17] Q. U. Nguyen, T. N. Nguyen, B. M. Le, and J. Ngu, "A multi-task learning framework for detecting web attacks," *IEEE Access*, vol. 7, pp. 101954–101972, 2019.
- [18] R. C. Staudemeyer, "Applying long short-term memory recurrent neural networks to intrusion detection," *South African Computer Journal*, vol. 56, no. 1, pp. 136–154, 2015.
- [19] X. Xie, Y. Chen, F. Bao, and C. Lim, "Security-aware transformer network for malware detection," in *Proc. IEEE Int. Conf. Cybersecurity*, 2022, pp. 1–8.
- [20] M. Abdelnabi, A. Krombholz, and M. Fritz, "VisualPhishNet: Zero-shot phishing website detection using visual similarity," in *Proc. ACM CCS*, 2020, pp. 1681–1698.
- [21] A. Singh and P. Verma, "Hybrid machine learning and rule-based web application firewall," in *Proc. Int. Conf. Advanced Computing and Communication Systems (ICACCS)*, 2021, pp. 634–639.
- [22] L. Chen and T. Wu, "Anomaly detection for zero-day web attacks using autoencoders," *Future Generation Computer Systems*, vol. 131, pp. 156–167, 2022.
- [23] S. Rahman and M. Islam, "Scalable intrusion detection with machine learning for distributed web security," *IEEE Trans. Network and Service Management*, vol. 19, no. 3, pp. 2547–2558, 2022.
- [24] N. Patel and D. Shah, "Kubernetes-native machine learning web traffic analysis for real-time attack visibility," *Computers & Security*, vol. 128, p. 103168, 2023.
- [25] J. Lopez and R. Martin, "Security analytics using Prometheus and Grafana for AI-powered monitoring," in *Proc. European Conf. Systems Engineering*, 2023, pp. 1–9.
- [26] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 1, pp. 1–39, Mar. 2012.
- [27] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [28] I. Csic, "HTTP CSIC 2010 Dataset for the Evaluation of Web Intrusion Detection Systems," Spanish National Research Council, Madrid, Spain, 2010.
- [29] G. Andresini, F. Palomba, and A. Appice, "INSOMNIA: Towards concept-drift robustness in network intrusion detection," in *Proc. ACM AISec*, 2021, pp. 111–122.
- [30] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proc. ACM ASIA CCS*, 2017, pp. 506–519.
- [31] Sriram V, Surothaaman R, and Pavithra M, "AI-Powered Advanced Web Application Firewall — Source Code Repository," GitHub, 2025. [Online]. Available: <https://github.com/Darkwebnew/Projectwork1.git>